Preguntas

1. ¿Por qué Bicicleta funcionó en mostrar() y en masEconomico() sin cambiar una sola línea de Flota?

R// Porque Bicicleta hereda de Vehiculo y utiliza los mismos métodos que están definidos en la clase principal. Gracias al polimorfismo, Flota puede trabajar con diferentes tipos de vehículos sin necesidad de cambiar su código.

2. ¿Por qué Vehiculo es abstracta y Moto no? ¿Qué pasaría si Moto también lo fuera?

R// Vehiculo es abstracta porque es una clase general que sirve como base para otros vehículos y no se necesita crear directamente un objeto de ella. Moto no es abstracta porque sí podemos crear objetos de tipo Moto. Si también fuera abstracta, no podríamos crear objetos directamente de esa clase.

3. ¿Por qué Bodega implementa Asegurable pero no hereda de Vehiculo?

R// Porque Bodega no es un vehículo, sino un lugar donde se guardan cosas. Sin embargo, puede tener un seguro, por eso implementa la interfaz Asegurable. Esto permite que tenga métodos relacionados con el aseguramiento sin tener que heredar de Vehiculo.

4. ¿Qué diferencia hay entre super(placa, capacidadKg) y super.describir()?

R// La diferencia es que super(placa, capacidadKg) se utiliza para llamar al constructor de la clase padre y darle los datos iniciales al objeto.

En cambio, super.describir() sirve para llamar al método describir() que está en la clase padre y utilizar lo que ya tiene definido.

5. Cierre del curso: explica con tus palabras qué es la POO y señala en tu código del taller un ejemplo de cada pilar.

R// La programación orientada a objetos es una forma de programar donde organizamos el código utilizando clases y objetos. Esto nos permite tener un código más ordenado, reutilizar información y hacer que los programas sean más fáciles de entender.